

Chapter 6 — Fixed-Size Arrays & Array-Based Containers

Memory management, C-style arrays, heap allocation, and building a custom array container

How to read these notes

NOT TESTED Content flagged by course directions as **not required for this class**. Still included for completeness / upper-level courses.

EXTRA Useful background, mainly for projects. Blue boxes = remarks. Green boxes = worked examples.

6.1 Introduction to Data Structures

- Two things to decide when solving a problem computationally: the **algorithm** and the **data structures** used to implement it.
- Algorithm** = a well-defined set of instructions that can be followed to solve a problem.
 - Time complexity (chapters 4-5) measures algorithm efficiency.
 - Performance depends not only on the implementation but also on the **data structure it runs on**.
- Data structure** = a collection of data + the relationships among them + the functions/operations that apply to those values.
 - Provides an **interface** defining how data is accessed.
 - Organizes data in memory so that this access is efficient.
- Container** = a data structure used to store other objects. It **manages the memory** used by the objects it holds.

Container categories in the C++ standard library

Category	Meaning	Examples (chapter introduced)
Sequence container	Elements can be accessed sequentially	<code>std::array<></code> (6), <code>std::vector<></code> (7), <code>std::list<></code> (8), <code>std::forward_list<></code> (8), <code>std::deque<></code> (9)
Associative container	Stores data in sorted order ; fast search in $\Theta(\log(n))$ time	<code>std::map<></code> , <code>std::multimap<></code> , <code>std::set<></code> , <code>std::multiset<></code> (all 18)
Unordered associative container	No defined order; usually implemented with hash tables ; average-case $\Theta(1)$ lookup	<code>std::unordered_map<></code> , <code>std::unordered_multimap<></code> , <code>std::unordered_set<></code> , <code>std::unordered_multiset<></code> (all 17)
Container adaptor	An interface adapted for a specific purpose; usually restricts the functionality of a standard container	<code>std::queue<></code> (9), <code>std::stack<></code> (9), <code>std::priority_queue<></code> (10)

- This chapter starts with the most basic sequential container: the **fixed-size array**.

6.2 Array Fundamentals

6.2.1 C-Style Arrays

- Array** = a **fixed-size** container of objects stored **contiguously in memory**; every element must have the **same type**.

```
int32_t arr[] = {0, 1, 5, 3}; // line 1: initializer list, size 4
arr[2] = 2; // line 2: modify element
int32_t* ptr = arr; // line 3: ptr = &arr[0] (array decays)
ptr = &arr[1]; // line 4
```

- Indexing:** operator[] accesses or modifies any element in $\Theta(1)$ time, because `arr[i]` does the same thing as `*(arr + i)` (pointer arithmetic + dereference), which is constant time.
- Array decay:** with a few exceptions (such as `sizeof()`), an array expression **decays into a pointer to its first element** when assigned or passed as a function argument. That is why `ptr = arr` stores `&arr[0]`.
- No bounds checking.** Error checking is entirely the programmer's responsibility. Automatic checking would add performance overhead, which is undesirable when speed matters.
- Fixed size.** The array above is non-dynamic: its size is fixed at compile time. To insert more elements you must reallocate the data into a bigger array elsewhere in memory.

Remark: Uniform initialization (C++11)

- Curly braces give one common initialization syntax for everything from primitives to large objects: `int32_t num{281};`
- Default initialization** leaves atomic types undefined; **value initialization** (empty braces) gives them an initial value:

Default Initialization	Value Initialization
<code>int32_t x;</code> → x is undefined	<code>int32_t x{};</code> → x is 0
<code>int32_t* y;</code> → y is undefined	<code>int32_t* y{};</code> → y is nullptr
<code>int32_t z[5];</code> → undefined data	<code>int32_t z[5]{};</code> → all 0's

- Braces forbid narrowing conversions.** `int32_t num = 281.57;` narrows (truncates) to 281, but `int32_t num{281.57};` is a **compiler error**. Same rule applies for arrays.

6.2.2 The STL Array (`std::array`)

NOT TESTED You do not need to know how to use `std::array<>` for this class (you may need it in upper-level classes).

- Provided by the `<array>` library: `std::array<TYPE, SIZE> arr;`
- Declaration forms: `std::array<int32_t, 4> arr = {0, 1, 5, 3};` or `std::array<int32_t, 4> arr{0, 1, 5, 3};`
- It is a **wrapper over the C-style array** that adds features the C array lacks:
 - Member functions (e.g. `.size()`) and iterators (chapter 11).
 - Can be treated like a fundamental type: direct assignment/copy with `operator=`, and **value semantics** (pass into / return from functions by value).
- Finding size — C array vs `std::array`:

```
// C-array: total bytes / bytes per element
int32_t c_arr[] = {0, 1, 5, 3};
size_t c_arr_size = sizeof(c_arr) / sizeof(c_arr[0]); // stores 4
// std::array: member function does the work
std::array<int32_t, 4> cpp_arr = {0, 1, 5, 3};
size_t cpp_arr_size = cpp_arr.size(); // stores 4
```

Function	Behavior
<code>.size()</code>	Returns the size of the array
<code>.empty()</code>	Returns a Boolean specifying whether the array is empty
<code>.front()</code>	Returns the first element
<code>.back()</code>	Returns the last element
<code>.begin()</code> / <code>.end()</code>	Random access iterator to the first element / one past the last element
<code>.cbegin()</code> / <code>.cend()</code>	Constant random access iterator to the first element / one past the last element
<code>.rbegin()</code> / <code>.rend()</code>	Reverse iterator to the last element / one before the first element
<code>.crbegin()</code> / <code>.crend()</code>	Constant reverse iterator to the last element / one before the first element

- Rule of thumb: if you need a fixed-size array in C++, prefer `std::array<>` over a C array (cleaner interface, more features). But a `std::vector<>` is usually the better choice anyway (chapter 7).

Remark: Correct loop bounds (applies to all arrays — bounds checking is your job)

- C++ arrays are **0-indexed**; the last valid index is `SIZE - 1`.

INCORRECT	CORRECT
<pre>for (size_t i = 0; i <= SIZE; ++i) { arr[i] = i; }</pre>	<pre>for (size_t i = 0; i < SIZE; ++i) { arr[i] = i; }</pre>
Using both <code>i</code> and <code>i + 1</code> : stop before <code>SIZE - 1</code> , since <code>i + 1 == SIZE</code> is not a valid index.	
<pre>for (size_t i = 0; i < SIZE; ++i) { arr[i] = arr[i + 1]; }</pre>	<pre>for (size_t i = 0; i < SIZE - 1; ++i) { arr[i] = arr[i + 1]; }</pre>
Accessing <code>i - 1</code> : start at 1 (so <code>i - 1</code> is valid) and use <code><=</code> to still visit every element.	
<pre>for (size_t i = 1; i < SIZE; ++i) { arr[i - 1] = i; }</pre>	<pre>for (size_t i = 1; i <= SIZE; ++i) { arr[i - 1] = i; }</pre>

6.3 Storing Multidimensional Data in Arrays

- A 1-D array of size 12: `int32_t arr_1D[12] = {0, 1, ..., 11};` — elements are contiguous (addresses 4 apart for 32-bit ints):

0	1	2	3	4	5	6	7	8	9	10	11
0x100	0x104	0x108	0x10c	0x110	0x114	0x118	0x11c	0x120	0x124	0x128	0x12c

- Declaring a fixed-size 2-D array: `int32_t arr_2D[num_rows][num_cols];`
- Example (3 rows × 4 columns): `int32_t arr_2D[3][4] = { {0,1,2,3}, {4,5,6,7}, {8,9,10,11} };`
- Access with `[]` twice: **first bracket = row index, second = column index**. `arr_2D[1][2]` gives 6.
- Because **memory chips are 1-dimensional**, a 2-D array is laid out in memory exactly like the 1-D array above, in **row-major order**.
- The compiler converts 2-D indices into a 1-D index behind the scenes.

Conversion formulas

```
1-D_index = 2-D_row_index × num_columns + 2-D_column_index
2-D_row_index = 1-D_index / num_columns (integer division — decimal truncated)
2-D_column_index = 1-D_index % num_columns
% is modulo: the remainder of a division. E.g. 11 % 5 = 1.
```

Example 6.1

A 2-D array has 14 rows and 11 columns. The element at row 7, column 9 is accessed. What is its 1-D index?
`1-D_index = 7 × 11 + 9 = 77 + 9 = 86`

Example 6.2

A 1-D array of size 126 stores 2-D data with 7 rows and 18 columns. What row/column is index 59?
row = $59 / 18 = 3$ (integer division) column = $59 \% 18 = 5 \rightarrow$ row 3, column 5.

- **Summary:** regardless of dimension, **all arrays look like 1-D arrays in memory**. Any n -dimensional array can be converted to a 1-D array using arithmetic.

6.4 Heap-Allocated Arrays

Why the stack is not enough

- **Stack-allocated arrays** are automatically deallocated when they go out of scope.
- **Limitation:** the size must be a **constant known at compile time**. This breaks if you only learn the size at runtime.

```
size_t size;
std::cin >> size;
int32_t arr[size];    // NOT allowed: size is a runtime value
```

- Some compilers allow it, but you should not: e.g. an absurdly large `size` would exceed available stack space and crash the program immediately.

Footnote

Variable-length stack arrays were permitted in C99 but discontinued in later versions. To use a variable for a stack array size, you must explicitly initialize it and declare it `constexpr` (meaning its value can be evaluated at compile time).

Using the heap

- The **heap** is memory that is **not managed automatically** — you are responsible for deallocating whatever you allocate.
- `new` allocates; `delete` deallocates. For arrays you must use `delete[]`.
- Failure to deallocate causes a **memory leak**: memory that is no longer accessible still remains on the heap.

```
size_t size;
std::cin >> size;
int32_t* arr = new int32_t[size];
/* ... do stuff with array ... */
delete[] arr;
```

Heap-allocated 2-D arrays

- Allocate the **outer dimension (rows) first**, then each **inner dimension (columns)**.

```
size_t rows, cols;
std::cout << "Enter the number of rows: " << std::endl;
std::cin >> rows;
std::cout << "Enter the number of columns: " << std::endl;
std::cin >> cols;
int32_t** arr = new int32_t*[rows];    // array of pointers to int arrays
for (size_t r = 0; r < rows; ++r) {
    arr[r] = new int32_t[cols];        // each row gets its own array
} // for r
int32_t val = 0;
for (size_t r = 0; r < rows; ++r) {    // rows in the OUTER loop (caching)
    for (size_t c = 0; c < cols; ++c) {
        arr[r][c] = val++;
    } // for c
} // for r
/* ... do stuff with the array ... */
for (size_t r = 0; r < rows; ++r) {
    delete[] arr[r];                  // delete rows FIRST
} // for r
delete[] arr;                          // then the array of pointers
```

- Structure: `arr` is a pointer to a heap array that **stores pointers to int arrays**; the loop initializes each row array with `cols` elements.
- **Deletion order is the reverse of allocation order.** Since `arr` was created before `arr[0]`, `arr[1]`, ..., you must delete all row arrays before deleting `arr`.
- **Caching:** when looping through a multidimensional array, loop the **outermost dimension (rows) in the outermost loop**. It is much more efficient to sequentially access items close together in memory than items far apart.
 - Loop rows, then columns $\rightarrow$ **faster** (memory accesses are contiguous).
 - Loop columns, then rows $\rightarrow$ **slower** (accesses are not contiguous).

Stack vs heap arrays — summary

Stack-allocated array	Heap-allocated (dynamic) array
Automatically deallocated when it goes out of scope	Programmer must deallocate before the program ends
Accessing an element needs only one memory access regardless of dimension (compiler computes the address)	Larger dimensions need more than one memory access (find the row pointer, then the element inside that row)
Limited in size; dimensions must be known at compile time $\rightarrow$ not very versatile	Flexible; use when the size cannot be determined until runtime

- Working with heap arrays is messy. `std::vector<>` abstracts this away: it gives all the functionality of a dynamically allocated array, handles allocation/deallocation for you, and can **grow** with the data (chapter 7).

Remark: dynamic memory discipline

- Whoever calls `new` is responsible for calling `delete`. Write the `delete` **immediately** after writing the `new`, before doing any other work, so you don't forget.
- After deleting a pointer, set it to `nullptr`:

```
delete ptr;
ptr = nullptr;
```

- This guards against unintentional use of a deleted value and against **double deletes** (`delete nullptr` is safe and does nothing, but calling `delete ptr` twice could crash your program).

Note: smart pointers

To avoid the hassle of `new/delete`, the `<memory>` library provides `std::unique_ptr<>`, `std::shared_ptr<>`, and `std::weak_ptr<>`. They are wrappers over normal pointers with enhanced functionality and automatic memory management (chapter 27).

6.5 Copying with Pointers

- Goal: copy from a **source** array into a **destination** array (assume the destination is large enough). You are given:

```
int32_t* first; // pointer to the first element of the source array
int32_t* last;  // pointer ONE PAST the last element of the source array
int32_t* dest;  // pointer to the first position of the destination array
```

- Procedure:
 - Dereference `first` and assign to `*dest` (`*dest = *first`).
 - Increment `first` to move to the next source element (`first++`).
 - Increment `dest` to move to the next destination position (`dest++`).
 - Repeat until `first` reaches `last`.

while loop	for loop
<pre>while (first != last) { *dest++ = *first++; }</pre>	<pre>for (; first != last; ++first, ++dest) { *dest = *first; }</pre>

- In the while loop, the **postfix** `++` means incrementing happens **after** the dereference and assignment.
- Use `first != last`, **not** `first < last`: only `!=` is guaranteed to work on all types of containers.
- This method extends across different containers using **iterators** (chapter 11). Pointers/iterators are preferable to indexing because they are more generic: the same code works for many container types, even ones that do not support indexing.

NOT TESTED `memcpy()` — you do not need to understand how it works for this class, but you will see it a lot in upper-level C++ courses.

```
void* memcpy(void* dest, const void* source, size_t num);
```

- Declared in `<cstring>`. Copies `num` **bytes** from `source` to `dest`.
- The underlying types are irrelevant — data is copied as **raw bytes**.
- Undefined behavior** if the objects overlap, or if either points to an invalid address or `nullptr`.
- A `void*` is a **generic pointer** with no associated type; it can reference an object of any data type.

```
int32_t src[] = {1, 2, 3, 4, 5};
int32_t dst[5];
std::memcpy(dst, src, sizeof(src));
for (size_t i = 0; i < 5; ++i) {
    std::cout << dst[i] << " "; // prints 1 2 3 4 5
} // for i
```

- `sizeof()` returns the number of bytes a type or object occupies. `sizeof(src)` is the total size of the array, so `memcpy()` copies the right number of bytes.

6.6 Sequential and Random Access

- Different containers support different **access methods** for iteration.
- Random access:** you can access any arbitrary element in a sequence **as efficiently as any other**, at any time, regardless of the size of the sequence.
 - Example: an **array** — any item is directly accessible in **$O(1)$** via `operator[]`.
 - User wants the 5th element → jumps to it directly in constant time.
- Sequential access:** elements are visited in a **predetermined order**, each accessed directly after its predecessor.
 - To reach an element, the container must be read **from the beginning** up to that element; there is no way to compute the memory location of a random element directly.
 - No guarantee that one element is as easy to reach as any other.
 - Example: a **linked list** (chapter 8) — without supporting data structures, reaching the n th element requires iterating through the first n elements.

6.7 Data Storage and Retrieval

What to store in a container: values vs pointers

- Storing by value:** the container has **full ownership** of the memory used for its data and is responsible for managing it.
 - Insertion/deletion should be requested from the container through an internal operation (e.g. an `insert()` member function).

- Most common choice, but **expensive to copy** if the stored objects are large.
- **Storing by pointer:** the container no longer directly owns the data and is not responsible for allocating/deallocating the values — only the pointers.
 - **Dangerous:** no direct protection — data can be modified elsewhere without the container's permission, and the container is easily **invalidated** if something else deletes what its pointers point to.
 - Useful for containers of C-strings (`char*`) and for **shared data** (one master container owns the data; smaller containers access it through pointers, so the data exists in memory only once).

What to Store	Value	Pointer
Example	<code>T data;</code>	<code>T* data;</code>
Data ownership	Container	Container or other object
Drawbacks	Large objects can take time to copy	Unsafe; can be modified elsewhere or invalidated
Usage	Most common	Used for <code>char*</code> , shared data

What to return from a container (e.g. from `operator[]`)

- **Return a copy (by value):** inefficient — a copy is made on every access, which is slow for large objects.
- **Return a pointer:** no copies needed, but **unsafe** — the user could modify anything, iterate off the end / access memory they shouldn't, or even delete the container's contents. Returning pointers to private data generally isn't ideal.
- **Return a reference: usually the best.** No copy (faster than by value) and safer than a pointer, because a reference cannot be modified to maliciously reach other memory.
 - To prevent the user from modifying the data, return a **const reference**.

What to Retrieve	Value	Pointer / Const Pointer	Reference / Const Reference
E.g.	<code>T get_elt(size_t idx);</code>	<code>T* get_elt(size_t idx);</code>	<code>T& get_elt(size_t idx);</code>
Notes	Costly for copying large objects	Unsafe; can be used to access prohibited memory	Usually a good choice; safer than pointers and faster than copies

6.8 Implementing an Array-Based Container

6.8.1 Blueprint of the custom `Array`

- Goal: a container that manages a dynamically allocated array and **abstracts memory work away from the user** — no manual reallocation to insert an element, no deep-copy bookkeeping to copy or assign.

```
template <typename T>
class Array {
    T* data;          // Array data (heap-allocated)
    size_t length;    // Array size
public:
    Array(size_t len = 0) : length{len} {
        data = (len ? new T[len] : nullptr);
    } // Array()

    size_t size() const {
        return length;
    } // size()
};
```

- `data` points to a heap-allocated array holding the container's data; `length` stores the number of elements.
- Constructor: the user passes `len`. `Array<int32_t> a{5};` → `length = 5`, `data = new T[5]`.
- Default argument: `Array<int32_t> b;` → `len = 0`, so `length = 0` and `data = nullptr` (that is what the ternary on the `data` line guarantees).
- `size()` is declared **const** because it does not modify any member variables.

The problem: shallow copies

```
Array<int32_t> a{5};
Array<int32_t> b = a;    // copies members one by one
```

- The `data` member is a **pointer**, so the pointer is copied — not the objects it points to. `a` and `b` end up **sharing the same memory**.
- Consequences: modifying `a` changes `b`; deallocating `a` deallocates `b`'s data too. Accessing `b` afterward reads invalid memory, and deallocating `b` afterward is a **double delete** that can crash the program.
- Fix: write custom copy/assignment operations that copy the **data objects**, not just the pointers.

6.8.2 The Big Three

- If a class manages **dynamic memory** as its data, define all three (the "Rule of Three"):
 1. **Destructor** — `~Array()`
 2. **Copy constructor** — `Array(const Array& other)`
 3. **Overloaded assignment operator** — `Array& operator=(const Array& rhs)`

NOT TESTED Remark: the "Big Five" (move semantics — see 6.9, not covered in this class)

- Two additional methods are often included: **4.** move constructor `Array(Array&& other)` **5.** move assignment operator `Array& operator=(Array&& rhs)`
- They **transfer ownership** of data from one object to another instead of copying it, assigning the copy, and discarding the original.
- `&&` denotes an **rvalue reference**: a temporary, unnamed object created during execution. Mnemonic: the "r" in rvalue = "right-hand side", since rvalues can never appear on the left of an assignment (they are temporary and often cannot outlive the statement that created them).

```
int32_t a = 10;      // '10' is an rvalue
int32_t b = 20;      // '20' is an rvalue
int32_t c = a + b;    // 'a + b' is an rvalue
int32_t d = a;        // 'a' is NOT an rvalue (named variable)
```

- Move operations **only** run when an **rvalue is present**; it is harmless to steal from a temporary that would be destructed anyway. If no rvalue is present, they are **not** invoked automatically, because non-temporaries may be used later.
- To move a named variable you must explicitly call `std::move()`:

```
Array<int32_t> a{5};
Array<int32_t> b = std::move(a); // a transfers ownership of its data to b
```

- This avoids a separate copy of `a`. After the move, `a` is in an **unspecified state** and should not be used until reassigned.

- In this chapter, the move operations are **disabled** with `= delete` (setting any member function to `delete` suppresses it and disallows it from being invoked):

```
template <typename T>
class Array {
    T* data;
    size_t length;
public:
    Array(size_t len = 0) : length{len} { data = (len ? new T[len] : nullptr); } // Constructor
    ~Array();                          // Destructor
    Array(const Array& other);          // Copy Constructor
    Array& operator=(const Array& rhs); // Overloaded Assignment Operator
    Array(Array&& other) = delete;      // Move Constructor (disabled)
    Array& operator=(Array&& rhs) = delete; // Move Assignment (disabled)
    size_t size() const { return length; }
};
```

6.8.3 The Destructor

```
~Array() {
    delete[] data; // delete dynamic memory owned by container
    data = nullptr; // safeguard against usage / double delete
} // ~Array()
```

- Both lines take 1 step (assuming `T` has no destructor of its own), so the destructor is **$\Theta(1)$** .

6.8.4 The Copy Constructor

- A copy constructor: **no return value, same name as the class**, and takes a **reference** to the object it copies from.
- The reference is essential: without it, a copy would have to be made **before** the copy constructor could run — which is not allowed.
- The body must copy each data element one by one (deep copy), not just the pointer.

Body version	Member-initializer-list version (cleaner)
<pre>Array(const Array& other) { data = new T[other.length]; length = other.length; for (size_t i = 0; i < length; ++i) { data[i] = other.data[i]; } // for i } // Array()</pre>	<pre>Array(const Array& other) : data{new T[other.length]}, length{other.length} { for (size_t i = 0; i < length; ++i) { data[i] = other.data[i]; } // for i } // Array()</pre>

- Every element must be visited → time complexity **$\Theta(n)$** , where n is the number of elements.

6.8.5 The Overloaded Assignment Operator

Attempt 1 — naive

```
Array& operator=(const Array& rhs) {
    delete[] data; // delete current data
    length = rhs.length;
    data = new T[length];
    for (size_t i = 0; i < length; ++i) { data[i] = rhs.data[i]; }
    return *this;
} // operator=()
```

- Fails on **self-assignment** (`a = a`) — the data is deleted before it is copied.

Attempt 2 — with a self-assignment check

```

Array& operator=(const Array& rhs) {
    if (this == &rhs) { return *this; } // compare memory addresses
    delete[] data;
    length = rhs.length;
    data = new T[length];
    for (size_t i = 0; i < length; ++i) { data[i] = rhs.data[i]; }
    return *this;
} // operator=()

```

- Works for most cases, but has **two problems**:
 1. The `this == &rhs` check runs on **every** assignment, even though self-assignment rarely happens.
 2. **Not exception safe**: if `new T[length]` fails, an exception is thrown and the operation terminates — but the old array was already deleted, so all original data is lost, `data` points to deleted memory, and `length` is wrong. The `Array` is **corrupted**.

Attempt 3 — the copy-swap method (best)

- Idea: make a temporary **deep copy** of the new `Array`, **swap** contents with the old `Array`, and let the old contents be destroyed automatically when the temporary goes out of scope.
- Benefits: **no self-assignment check needed** (cleaner code) and the **original data is retained** until the assignment succeeds. If allocation fails, the failed operation has no side effects on existing data — this is called a **strong exception guarantee**.

```

void swap_array(Array& other) {
    std::swap(data, other.data);
    std::swap(length, other.length);
} // swap_array()

Array& operator=(const Array& rhs) {
    Array temp{rhs}; // deep copy (uses the copy constructor)
    swap_array(temp); // old data now in temp; new data now in this
    return *this;    // reference to the Array assigned to
} // operator=()
// temp goes out of scope -> old data cleaned up by the destructor

```

Walkthrough of `arr1 = arr2`

- Start: `arr1 = {1,2,3,4}`, length 4 | `arr2 = {5,6,7,8,9}`, length 5
 - **Copy line**: a deep copy of `arr2` is made into local `temp` → `temp = {5,6,7,8,9}`, length 5
 - **Swap line**: contents of `arr1` and `temp` are swapped → `arr1 = {5,6,7,8,9}` length 5; `temp = {1,2,3,4}` length 4
 - **Return line**: `*this` (i.e. `arr1`) is returned; when the function ends, `temp` (holding `arr1`'s old data) is destroyed by the `Array` destructor.
 - Self-assignment `arr1 = arr1` simply swaps the `Array` with a copy of its own data — harmless.
- Because the assignment operator does a deep copy, its complexity is $\Theta(n)$, where n is the size of `rhs`.

6.8.6 The Subscript Operator

- Return a **reference** (faster than copies, safer than pointers).
- Check validity of the index; if invalid, **throw** a `std::runtime_error` (we don't want to return a reference the user could unintentionally modify).

Non-const version	Const version
<pre> T& operator[](size_t idx) { if (idx < length) { return data[idx]; } // if throw std::runtime_error {"Invalid index provided to operator[]"}; } // operator[]() </pre>	<pre> const T& operator[](size_t idx) const { if (idx < length) { return data[idx]; } // if throw std::runtime_error {"Invalid index provided to operator[]"}; } // operator[]() </pre>

Why both versions are required

```

Array<int> a{5};
for (size_t i = 0; i < a.size(); ++i) { a[i] = i; std::cout << a[i] << " "; } // prints 0 1 2 3 4

const Array<int> b = a;
for (size_t i = 0; i < b.size(); ++i) { std::cout << b[i] << " "; } // COMPILER ERROR

```

```

error: passing 'const Array' as 'this' argument discards qualifiers [-fpermissive]

```

- A **const object cannot call a non-const member function**. Since `b` is a `const Array`, it cannot call the non-const `operator[]`.
- The **non-const** version is invoked for a non-const `Array`; the **const** version is invoked for a `const Array`. Whether the call actually modifies anything is irrelevant — only the constness of the **object** decides.

Remark: defining member functions outside the class

- All functions above were written **inside** the class definition. To define one outside, use the **scope resolution operator** `::` (with template declarations as needed). It goes **after the return type, before the function name**.

```
template <typename T>
class Array {
    T* data;
    size_t length;
public:
    Array(size_t len = 0);
    ~Array();
    Array(const Array& other);
    void swap_array(Array& other);
    Array& operator=(const Array& rhs);
    T& operator[](size_t idx);
    const T& operator[](size_t idx) const;
    size_t size() const;
};

template <typename T>
Array<T>::Array(size_t len) : length{len} { data = (len ? new T[len] : nullptr); } // Array()

template <typename T>
Array<T>::~~Array() { delete[] data; data = nullptr; } // ~Array()

template <typename T>
Array<T>::Array(const Array& other) : data{new T[other.length]}, length{other.length} {
    for (size_t i = 0; i < length; ++i) { data[i] = other.data[i]; } // for i
} // Array()

template <typename T>
void Array<T>::swap_array(Array& other) { std::swap(data, other.data); std::swap(length, other.length); }

template <typename T>
Array<T>& Array<T>::operator=(const Array& rhs) { Array temp{rhs}; swap_array(temp); return *this; }

template <typename T>
T& Array<T>::operator[](size_t idx) {
    if (idx < length) { return data[idx]; }
    throw std::runtime_error{"Invalid index provided to operator[]"};
} // operator[]()

template <typename T>
const T& Array<T>::operator[](size_t idx) const {
    if (idx < length) { return data[idx]; }
    throw std::runtime_error{"Invalid index provided to operator[]"};
} // operator[]()

template <typename T>
size_t Array<T>::size() const { return length; } // size()
```

6.8.7 Implementing Insert

- Insertion is complicated for **two** reasons:
 1. Elements in an array-based container must be stored **contiguously** in memory.
 2. Once a data array is allocated, its **size cannot change**; more space means allocating a new array and moving the data.
- Inserting 2 between 1 and 3 in {1, 3, 4, 5}: every element after the insertion point must be **shifted one to the right** to make space.

1 3 4 5 → 1 3 4 5 → 1 2 3 4 5

```
template <typename T>
bool insert(size_t idx, T val) {           // e.g. insert(1, 2) puts 2 at index 1
    if (idx < length) {
        for (size_t i = length - 1; i > idx; --i) {
            data[i] = data[i - 1];         // shift right
        } // for i
        data[idx] = val;
        return true;                       // inserted successfully
    } // if
    return false;                           // not inserted
} // insert()
```

- Complexity (assuming `idx` is valid, n elements):
 - **Best case $\Theta(1)$** — insert at the very **end** (no shifting needed).
 - **Worst case $\Theta(n)$** — insert at the very **beginning** (all elements shift).
 - **Average case $\Theta(n)$** — on average $n/2$ elements shift, and $\Theta(n/2) = \Theta(n)$ after dropping coefficients.
- But this still does not work in all cases: if `data` holds 4 elements, that array **cannot expand**. Adding a 5th element requires allocating a larger array elsewhere and moving the data.

Growing the array — naive (+1 each time)

- Steps: allocate a new array of size 5 → copy the original 4 elements → deallocate the old array and reset the pointer → add the 5th element.
- **Not performant**: copying is $\Theta(n)$ and would happen on **every** insertion past capacity.
 - Example: with 1,000 elements, adding the 1,001st and 1,002nd requires **2 array allocations and 2,001 copies** (1,000 on the first allocation, 1,001 on the second).

Growing the array — doubling (the right way)

- Plan ahead: on reallocation, **double** the capacity instead of adding 1 (e.g. 4 → 8, not 4 → 5).
- Then the 6th, 7th, and 8th elements need no copying. Adding the 9th reallocates and copies, but capacity doubles to 16, so elements 10–16 are inserted in constant time with no copying overhead.
- Because more space is allocated than needed, **size and capacity differ**, so both must be tracked:
 - **capacity** tells you when a reallocation is required (maximum elements the underlying array can hold).
 - **size** tells you the index of the last valid element (e.g. with capacity 8 but 5 valid elements, the user must not access indices 5–7).

```
template <typename T>
class Array {
    T* data;           // pointer to Array data
    size_t size;        // number of valid elements in 'data' array
    size_t capacity;    // capacity of underlying 'data' array
    ...
};
```

- **Why not triple or quadruple?** You could, but it wastes memory: there is no guarantee all that space is used, and the performance improvement is **asymptotically insignificant**.
- Remaining features (deletion, manually changing capacity) are not implemented here: they get complicated (insertion and deletion both require shifting), and it isn't necessary — C++ already provides `std::vector<>`, which does all of this and more (chapter 7). No need to reinvent the wheel.

6.9 Move Semantics

NOT TESTED The entirety of 6.9 is not taught in this class. It is core C++ knowledge worth reading after the class. Move semantics are not banned on the autograder — you may use them in assignments if you understand them, but they are not required.

- **Move semantics** allow resources to be **transferred** between objects instead of copied, avoiding unnecessary copies of temporaries.

6.9.1 Lvalues and Rvalues

- **Lvalue** ("left-hand" value, also **locator value**): a **named** object with an identifiable location in memory.
- **Rvalue** ("right-hand" value): a **temporary, unnamed** object that cannot be assigned to.
- In `int32_t x = 281;` → `x` is an lvalue, `281` is an rvalue.
- Lvalues can be assigned from rvalues or other lvalues; rvalues can never be assigned to.

```
int32_t x = 281;
int32_t y = x;    // OK, x and y are both lvalues
281 = x;          // NOT OK, 281 is an rvalue
```

- A function returning **by value** returns an **rvalue**; a function returning **by reference** returns an **lvalue reference**, which can be assigned to.

Returns by value → rvalue	Returns by reference → lvalue reference
<pre>int32_t get_favorite_class() { static int32_t fav_class = 281; return fav_class; } int main() { int32_t x = get_favorite_class(); // OK get_favorite_class() = 370; // NOT OK }</pre>	<pre>int32_t& get_favorite_class() { static int32_t fav_class = 281; return fav_class; } int32_t main() { get_favorite_class() = 370; // OK }</pre>

- You cannot construct an lvalue reference from an rvalue: `int32_t& x = 281;` is not OK.
- **Exception:** a **const** lvalue reference can bind to an rvalue: `const int32_t& x = 281;` is OK.

```
void foo(std::string& s)      { ... } // non-const lvalue ref: lvalues only
void bar(const std::string& s) { ... } // const lvalue ref: lvalues AND rvalues
void baz(std::string&& s)     { ... } // rvalue ref: rvalues only

std::string str1 = "EECS"; std::string str2 = "281";
foo(str1);         // OK (lvalue)      bar(str1);         // OK
foo(str2);         // OK (lvalue)      bar(str2);         // OK
foo(str1 + str2);  // NOT OK (rvalue)   bar(str1 + str2);  // OK (param is const)
foo("EECS 281");   // NOT OK (rvalue)   bar("EECS 281");   // OK (param is const)

baz(str1);         // NOT OK (lvalue)
baz(str1 + str2);  // OK (rvalue)
baz("EECS 281");   // OK (rvalue)
```

- **Why this matters:** with an lvalue you must be careful, since something else may still need the data. With an rvalue you know the object is temporary and can be "abused" freely — enabling optimizations like **stealing its resources**.
- An **rvalue reference** is denoted `&&` and binds only to temporaries. You can overload a function on it:

```
void qux(const std::string& s) { /* runs for lvalues */ }
void qux(std::string&& s)     { /* runs for rvalues; can safely steal from s */ }

qux(str1);           // first version (const&)
qux(str1 + str2);    // second version (&&)
qux("EECS 281");     // second version (&&)
```

- Even though rvalues *can* bind to a const lvalue reference, they **always** go to the `&&` overload when one exists.

6.9.2 Implementing the Move Constructor

```

class String {
    char* data; // pointer to char array on heap
    size_t length;
public:
    String(const char* str) { // constructor from char*
        length = strlen(str);
        data = new char[length + 1];
        std::cout << "Char Ctor: Heap allocation made" << std::endl;
        for (size_t i = 0; i < length; ++i) { data[i] = str[i]; }
        data[length] = '\0';
    }
    String(const String& other) // copy constructor
        : data{new char[other.length + 1]}, length{other.length} {
        std::cout << "Copy Ctor: Heap allocation made" << std::endl;
        for (size_t i = 0; i < length; ++i) { data[i] = other.data[i]; }
        data[length] = '\0';
        std::cout << "Copy Ctor: String successfully copied" << std::endl;
    }
};
class Student {
    String name;
public:
    Student(const String& name_in) : name{name_in} {}
};
int main() { Student s{"Alice"}; }

```

```

Output: Char Ctor: Heap allocation made
        Copy Ctor: Heap allocation made
        Copy Ctor: String successfully copied

```

- **What went wrong:** "Alice" is first converted into a temporary String rvalue (heap allocation #1). That rvalue is passed to the constructor and assigned to name, invoking the **copy constructor** (heap allocation #2 + a copy). The rvalue is then destructed. Two allocations and a copy where one allocation would do.
- **Fix: a move constructor** — a constructor accepting an rvalue of the same type, invoked when an object is initialized from an rvalue. It does not allocate; it takes ownership of the rvalue's data.

```

String(String&& other) noexcept
    : data{other.data}, length{other.length} { // shallow copy = "steal"
    other.data = nullptr; // CRITICAL
    other.length = 0;
    std::cout << "Move Ctor: Data transferred from rvalue" << std::endl;
} // String()

```

- Setting other.data = nullptr is essential: the rvalue is destructed right after, and without this the stolen data would be deleted with it.
- **Remark:** move constructors and move assignment operators should be declared noexcept, indicating they do not throw exceptions.
- Adding a Student(String&& name_in) : name{name_in} {} overload alone **changes nothing** — the output is still two allocations. Reason: inside the function, name_in has a **name and an identifiable address**, so an rvalue reference parameter **behaves like an lvalue**.
- You must explicitly cast it back with std::move():

```

class Student {
    String name;
public:
    Student(const String& name_in) : name{name_in} {} // lvalues
    Student(String&& name_in) : name{std::move(name_in)} {} // rvalues
};

```

```

Output: Char Ctor: Heap allocation made
        Move Ctor: Data transferred from rvalue

```

- **Remark:** std::move() **casts** its argument to an rvalue; it does not physically move anything. It tells the compiler the object can be treated as an rvalue and safely stolen from (the actual stealing happens in the move constructor / move assignment operator). After a move, the source is in an **unspecified state** and must not be used until reassigned.

6.9.3 Implementing the Move Assignment Operator

- Same idea as the move constructor: overload operator= to take an rvalue reference and steal its data.

```

String& operator=(String&& rhs) noexcept {
    if (this == &rhs) { return *this; } // self-assignment check
    delete[] data; // delete original data
    length = rhs.length; // steal length and pointer
    data = rhs.data;
    rhs.length = 0; // clear rvalue so stolen data isn't deleted
    rhs.data = nullptr;
} // operator=()

```

- As with copy-swap, the self-assignment check can be removed by swapping instead:

```

String& operator=(String&& rhs) noexcept {
    std::swap(this->length, rhs.length);
    std::swap(this->data, rhs.data);
    return *this;
} // operator=()

```

Remark: how `std::swap()` is implemented

Before C++11 (copies)	C++11 and later (moves)
<pre>template <typename T> void swap(T& a, T& b) { T temp = a; a = b; b = temp; } // swap()</pre>	<pre>template <typename T> void swap(T& a, T& b) { T temp = std::move(a); a = std::move(b); b = std::move(temp); } // swap()</pre>

Move construction and move assignment transfer ownership instead of performing unnecessary copies.

6.9.4 Return Value Optimization (RVO)

- **Do not** use `std::move()` to return a local object — it **suppresses RVO**.
- If a function returns a local object **by value** and the object's type **matches the return type**, the compiler may construct the object **directly at its destination** in memory. No copy or move happens at all. Because the copy/move is elided, RVO is formally a **copy elision** optimization.

Good (RVO applies)	Bad (RVO suppressed)
<pre>std::string gen_big_string(size_t n) { std::string big_string; /* ... generate ... */ return big_string; } int main() { std::string str = gen_big_string(281); }</pre>	<pre>std::string gen_big_string(size_t n) { std::string big_string; /* ... generate ... */ return std::move(big_string); } int main() { std::string str = gen_big_string(281); }</pre>

- With `std::move()` you return an **rvalue reference** (`std::string&&`), not a local object of the return type, which violates RVO's conditions. The compiler must then build a separate instance in `gen_big_string()`'s stack frame and transfer its data into `str` in `main()`'s stack frame — slower.

6.9.5 Perfect Forwarding and Forwarding References

- **Perfect forwarding**: a templated function accepts a **forwarding reference** (also written `&&`) and uses `std::forward<>` to **preserve the original value category** (lvalue vs rvalue) of its argument.

```
template <typename T>
void foo(T&& arg) {
    bar(std::forward<T>(arg));
} // foo
```

- If `arg` arrives as an lvalue reference it is passed to `bar()` as an lvalue reference; if it arrives as an rvalue reference it is passed as an rvalue reference.
- **Why `std::forward<>` is needed**: all function parameters behave like **lvalues** (the parameter itself is a named object), so `arg` is always an lvalue inside the function. `std::forward<>` performs a **conditional cast** based on the original value category: nothing for an lvalue, cast to rvalue for an rvalue.
- It works via **reference collapsing** during template deduction:
 - `T& &` → `T&` (lvalue reference)
 - `T& &&` → `T&` (lvalue reference)
 - `T&& &` → `T&` (lvalue reference)
 - `T&& &&` → `T&&` (rvalue reference)
- So with `template <typename T> void foo(T&& arg);`: passing an lvalue reference of type `Thing` deduces `foo(Thing& &&) → foo(Thing&)`; passing an rvalue reference deduces `foo(Thing&& &&) → foo(Thing&&)`.
- **Value**: avoids excessive copying and lets one method support many input value types without writing multiple overloads. Best use case: an object moved through **multiple function calls** between creation and destination.
- Used widely in the standard library — e.g. `std::vector<>::emplace_back(Args&&... args)`, which forwards constructor arguments to the object's constructor so it is built directly at the back of the vector (chapter 7).

6.10 Valgrind and Memory Debugging Tools

- Manual memory management risks **segmentation faults, uninitialized values, and memory leaks**. **valgrind** detects these.
- Run it as: `valgrind ./exec` (where `exec` is your compiled executable).

Example 1 — off-the-end access, use-after-free, and a leak

```
// valgrind.cpp
1 int main() {
2     int32_t* arr = new int32_t[3];
3     for (size_t i = 0; i <= 3; ++i) {
4         arr[i] = i * i;
5         std::cout << arr[i] << " ";
6     } // for i
7     delete[] arr;
8     int32_t val;
9     arr[0] = val;
10    int32_t* leak = new int32_t[100];
11 } // main()
```

- The program prints "0 1 4 9" and **looks** fine, but valgrind reports 3 errors from 3 contexts.

valgrind output (key lines)	Translation
Invalid write of size 4 at main (valgrind.cpp:4) Address ... is 0 bytes after a block of size 12 alloc'd at operator new[](unsigned long) by main (valgrind.cpp:2)	Line 4 wrote 4 bytes to an invalid address, past the end of a 12-byte block that was allocated on line 2 (3 ints × 4 bytes).
Invalid read of size 4 at main (valgrind.cpp:5) Address ... is 0 bytes after a block of size 12 alloc'd by main (valgrind.cpp:2)	Line 5 read 4 bytes past the end of that same block. Cause: <code>i</code> runs off the end — <code>arr[3]</code> is invalid. Fix: change <code><=</code> to <code><</code> on line 3.
Invalid write of size 4 at main (valgrind.cpp:9) Address ... is 0 bytes inside a block of size 12 free'd at operator delete[](void*) by main (valgrind.cpp:7) Block was alloc'd at by main (valgrind.cpp:2)	Line 9 wrote to memory that was already deallocated . Reading bottom-up: allocated on line 2, deleted on line 7, written on line 9 — a write to memory we no longer have access to.

HEAP SUMMARY:
 in use at exit: 400 bytes in 1 blocks
 total heap usage: 3 allocs, 2 frees, 73,116 bytes allocated

LEAK SUMMARY:
 definitely lost: 400 bytes in 1 blocks
 indirectly lost: 0 bytes in 0 blocks
 possibly lost: 0 bytes in 0 blocks
 still reachable: 0 bytes in 0 blocks
 suppressed: 0 bytes in 0 blocks
 Rerun with --leak-check=full to see details of leaked memory

- **definitely lost** — the pointer to that block is fully lost before the memory is deallocated. **Red flag.**
- **indirectly lost** — the pointer to that block lived inside another block that was definitely lost. **Red flag.**
- **still reachable** — heap memory not explicitly freed at exit, but the program still held a pointer/reference to it. Usually **not harmful** during development; the OS completes the cleanup. Not a true leak.
- Here 400 bytes were definitely lost: line 10 allocated 100 ints × 4 bytes = 400 bytes, never freed with `delete[]`.

Example 2 — returning a pointer to a local variable

```
1 int32_t* add_100(int32_t n) {
2     int32_t res = 100;
3     res = 100 + n;
4     int32_t *addr = &res;
5     return addr;
6 } // add_100()
8 void print_number(int32_t num) {
9     std::cout << num;
10 } // print_number()
12 int main() {
13     int32_t eecs = 281;
14     int32_t *sum = add_100(eecs);
15     print_number(*sum);
16 } // main()
```

Conditional jump or move depends on uninitialised value(s)
 ... by print_number(int) (valgrind.cpp:9)
 ... by main (valgrind.cpp:15)
Use of uninitialised value of size 8
 ... by print_number(int) (valgrind.cpp:9)
 ... by main (valgrind.cpp:15)

- **How to read messy output:** ignore the stream-internals lines and look for the entries that reference **your own file and line numbers**.
- All errors trace to line 15 calling `print_number()` with `*sum`, which reaches `std::cout` on line 9 — an **uninitialized value** is being passed, hinting that `sum` is uninitialized.
- **Cause:** `add_100()` returns `addr`, a pointer to a variable allocated **on the stack**, which goes out of scope when the function exits. The pointer stored in `sum` is therefore out of scope / uninitialized.

Remark: `--track-origins=yes`

```
./valgrind --track-origins=yes ./<executable_name>
```

Adds output such as:

```
Uninitialised value was created by a stack allocation
  at main (valgrind.cpp:14)
```

pinpointing that the uninitialized value (the pointer `sum`) was created on the stack on line 14.